

PROBLEM 3

Recommendation Systems

Predicting what users will like before they search for it

ML Interview Preparation Guide

What Is a Recommendation System?

Every time you open Netflix, it shows you movies you might like. Every time you open Amazon, it shows "customers also bought." Every time you open Spotify, it builds you a playlist. A recommendation system predicts what a user will like, before they search for it.

Netflix estimates their recommendation system saves them \$1 billion per year in customer retention. This is one of the highest-value ML applications in industry.

Real World Examples: Netflix ("Because you watched Inception..."), Amazon ("Customers who bought this also bought..."), Spotify Discover Weekly, TikTok infinite scroll, LinkedIn ("People you may know").

Step 0: Clarify the Problem

- **What are we recommending?** Movies? Products? Friends? Songs?
- **What is the goal?** Maximize clicks? Watch time? Purchases? These lead to very different systems!
- **How many users and items?** 100M users × 10M items needs serious engineering.
- **Explicit or implicit feedback?** Ratings and likes vs. clicks and watch time.
- **Cold start problem?** What do we show new users with no history? New items with no interactions?
- **Real-time or batch?** Does the recommendation update instantly (TikTok) or daily (email digest)?

Step 1: Types of Feedback Data

Explicit Feedback

The user directly tells you their preference: star ratings, thumbs up/down, written reviews. Clear signal but rare — Netflix found less than 5% of users rate movies.

Implicit Feedback

You infer preference from behavior: watch 90% of a movie = liked it, watch 10% and stopped = did not like it, click = mild interest, rewatch = loved it. Abundant and automatic but noisy — watching 90% might mean they fell asleep.

In the real world, almost all recommendation systems are built on implicit feedback because explicit feedback is too sparse. Implicit feedback requires careful interpretation.

Step 2: Exploring the Data

The User-Item Matrix

The core data structure is a matrix where each row = one user, each column = one movie, and each cell = rating or interaction (empty if no interaction). We want to predict the empty cells — that is literally what a recommendation system does.

The Sparsity Problem

In reality, this matrix is extremely sparse. Netflix has 200M users × 6000 movies. A typical user watches maybe 50 movies — only 50 out of 6000 cells filled. Sparsity greater than 99% — more than 99% of cells are empty. This is the fundamental challenge.

Long Tail Distribution

A tiny fraction of movies get most of the interactions (blockbusters). Most movies have very few ratings. This is a power law distribution. Easy to recommend popular movies (lots of data). Hard to recommend niche movies (very little data). But niche recommendations are the most valuable — users discover something they would never find themselves.

Position Bias — Critical Problem with Click Data

Users click on top results partly because they are at the top, not just because they are relevant. Position 1 gets 35% click rate. Position 10 gets only 2%. If you naively train on clicks, your model learns "position 1 things are relevant" — a self-fulfilling prophecy.

Fix: Inverse Propensity Scoring (IPS). Weight each click by the inverse of how likely that position is to be clicked. A click from position 5 counts much more than from position 1.

Step 3: Feature Engineering

User Features

- `user_age`, `user_location`, `user_device`
- `account_age_days`
- `avg_session_length`
- `favorite_genre` (inferred from behavior)
- `active_hours` (morning watcher vs. late night?)

Item Features

- **genre**: [Action, Sci-Fi, Drama]
- **director, cast** (top actors)
- **release_year, runtime_minutes, language**
- **average_rating_across_all_users**
- **popularity_score** (number of views)

Interaction Features (Most Powerful)

- **watch_percentage** — how much of the movie did they watch?
- **time_to_first_play** — did they click immediately or come back later?
- **number_of_sessions** — did they watch across multiple sittings?
- **device_type** — mobile vs. TV vs. laptop
- **time_of_day_watched** — context matters

Step 4: Statistics and Math

Similarity Measures

Cosine Similarity

Imagine each user as a vector — a list of their ratings. Cosine similarity measures the angle between two user vectors.

- $\text{cosine_similarity}(A, B) = (A \cdot B) / (||A|| \times ||B||)$
- Result = 1: identical taste
- Result = 0: completely different taste
- Result = -1: opposite taste

Pearson Correlation

Similar to cosine similarity but centers the data first (subtracts each user's mean rating). This handles harsh raters (always give 2-3 stars) vs. generous raters (always give 4-5 stars). More robust for explicit rating data.

Jaccard Similarity (for Binary Data)

When you only know if someone interacted or not (clicked vs. not clicked):

- $\text{Jaccard}(A, B) = |A \cap B| / |A \cup B|$
- = items both users liked / items at least one user liked

Matrix Factorization — The Core Math

We have a huge sparse user-item matrix R . We want to predict the missing values. Decompose R into two smaller matrices: $R \approx P \times Q^T$ where $P = (\text{Users} \times k)$ and $Q = (\text{Items} \times k)$ and k = number of latent factors.

Latent factors are hidden dimensions explaining why users like what they like. For movies: how action-oriented, how romantic, how cerebral. The predicted rating = dot product of user vector and movie vector.

You find P and Q by minimizing prediction error: $\text{Loss} = \sum (\text{actual} - P_{\text{user}} \cdot Q_{\text{movie}}^T)^2 + \lambda(\|P\|^2 + \|Q\|^2)$. Use gradient descent. The λ term is regularization — prevents overfitting by penalizing very large values.

Step 5: Models — From Simple to Complex

Approach 1: Collaborative Filtering

User-Based Collaborative Filtering

Find users most similar to the target user using cosine similarity. Look at what those similar users liked that the target user has not seen. Recommend those items. Intuition: "Users like you loved this."

- **Pros:** Simple, interpretable
- **Cons:** Does not scale — computing similarity between 100M users is extremely slow. Cold start problem.

Item-Based Collaborative Filtering

Find items most similar to what the user already liked. Recommend those similar items. More scalable than user-based because items are more stable than users and there are fewer items. "Because you watched X" is easy to explain.

- **Pros:** More scalable and stable than user-based
- **Cons:** Cold start problem for new items

Approach 2: Matrix Factorization (SVD-based)

SVD (Singular Value Decomposition): $R = U \times \Sigma \times V^T$. In practice, use Truncated SVD or learn P and Q via gradient descent.

- **ALS (Alternating Least Squares):** Fix Q , solve for P analytically. Fix P , solve for Q . Alternate until convergence. Great for distributed computing.
- **SGD (Stochastic Gradient Descent):** For each known rating, nudge P and Q in the right direction. Simpler to implement.
- **Pros:** Handles sparsity beautifully, captures latent structure
- **Cons:** Cold start for new users/items, hard to add side features

Approach 3: Content-Based Filtering

Recommend items similar to what the user liked, based on item features — not other users behavior. Build a user profile from their liked items (genres, directors, actors) and find similar items.

- **Pros:** No cold start for new items, explainable, no need for other users data
- **Cons:** Filter bubble — only recommends more of the same. No serendipity. Needs good item metadata.

Approach 4: Hybrid Systems

The real world uses a combination. Netflix, Spotify, Amazon all use hybrid approaches. Final score = $\alpha \times \text{collaborative_score} + \beta \times \text{content_score} + \gamma \times \text{popularity_score}$. This gives you collaborative filtering's power to discover surprising matches plus content-based's ability to handle cold start.

Approach 5: Deep Learning Models

Neural Collaborative Filtering (NCF)

Replaces the dot product in matrix factorization with a neural network. The dot product can only capture linear relationships. A neural network captures non-linear interactions between user and item embeddings.

Two-Tower Model — Industry Standard

Used by YouTube, Google, Pinterest. One neural network for users, one for items. Both produce embedding vectors of the same size. Score = cosine similarity between user vector and item vector.

The key advantage: compute all item vectors offline and store them. At serving time, only compute the user vector (fast), then find closest item vectors using Approximate Nearest Neighbor (ANN) search (FAISS library). This lets you search through millions of items in milliseconds.

- **Pros:** Scales to billions of users and items, very fast at serving time
- **Cons:** The two towers do not talk until the very end — misses complex feature interactions

Transformer-Based Models (BERT4Rec, SASRec)

Treat a user's watch history as a sequence and use a Transformer to predict the next item they will like. The self-attention mechanism lets the model focus on the most relevant past items — what you watched last week may matter more than 2 years ago.

- **Pros:** Captures sequential patterns and long-range dependencies, state of the art accuracy
- **Cons:** Computationally expensive, needs lots of data

The Real System: Multi-Stage Architecture

Real recommendation systems are NEVER a single model. They have multiple stages:

1. **RETRIEVAL**: Quickly narrow 10M items down to ~1000 candidates. Use Two-Tower model + ANN search. Must be very fast (milliseconds). Trade accuracy for speed.
2. **RANKING**: Precisely score and rank the 1000 candidates. Use deep neural network with rich features. Can be slower since you only score 1000, not millions.
3. **RE-RANKING / BUSINESS RULES**: Apply diversity, freshness, business constraints. Remove watched items, boost new releases, inject sponsored content.

Why staged? You cannot run a heavy neural network on 10M items in real-time. First stage trades accuracy for speed. Second stage trades speed for accuracy on a small set.

Step 6: Evaluation

Offline Metrics

- **Precision@K**: Of the top K recommendations, how many did the user actually like?
- **Recall@K**: Of all items the user liked, how many made it into the top K?
- **NDCG@K**: Cares about order — showing a relevant item at position 1 is worth more than position 10. $DCG = \sum \text{relevance}_i / \log_2(\text{position}_i + 1)$. $NDCG = DCG / IDCG$ (ideal ranking).
- **Hit Rate@K**: For what fraction of users did at least one relevant item appear in the top K?

Online Metrics (What Really Matters)

- **Click-Through Rate (CTR)**: What % of recommendations get clicked?
- **Watch time / Engagement time**
- **Conversion rate** (for e-commerce — did they buy?)
- **Diversity**: Are recommendations varied or all the same genre?
- **Long-term retention**: Do users come back next week?

CTR can be gamed — showing clickbait thumbnails gets high CTR but users are disappointed. Netflix optimizes for watch time, not clicks, because it better reflects genuine enjoyment.

Step 7: Production

The Cold Start Problem

New user cold start: ask onboarding questions, show popular/trending items as fallback, use demographic info. After just 5-10 interactions, switch to personalized recommendations.

New item cold start: use content-based features immediately (genre, director, cast). Boost new items artificially to gather early interaction data.

Explore vs. Exploit

Pure exploitation → filter bubble: user only sees same content forever → gets bored → churns.
Pure exploration → user sees random stuff they hate → bad experience. Multi-Armed Bandit algorithms solve this.

- **ϵ -Greedy:** With probability ϵ (say 10%), recommend a random item. Otherwise recommend the best known item.
- **Thompson Sampling:** Maintain a probability distribution over each item's quality. Sample from it — naturally explores uncertain items more.

Monitoring

- **CTR drift:** Did click rate suddenly drop? Maybe a bug in retrieval.
- **Coverage:** Are you recommending diverse items, or only top-100 popular ones?
- **Feedback loop detection:** If you only show Action movies → user only watches Action → model thinks they love Action → shows more Action → vicious cycle. Monitor genre distribution in recommendations.

How to Answer in an Interview

Start by clarifying what we are optimizing for — clicks, watch time, or purchases — because that changes everything. The core challenge is the sparse user-item matrix. Build a multi-stage system: Two-Tower model for fast candidate retrieval across millions of items using ANN search, then a heavier ranking model with rich features. For cold start, use content-based features for new items immediately and popularity-based fallback for new users. Build in exploration via epsilon-greedy or Thompson Sampling to avoid filter bubbles. Offline evaluation uses NDCG@K. The real test is A/B testing watch time and 30-day retention. User embeddings are precomputed in Redis. FAISS index updated nightly. Monitor genre diversity and popularity bias to catch feedback loops.